

aspects (i.e., structural software quality).

In general, you can view software quality from two perspectives: internal quality and external quality. The aspects of software that concern the users of the software—such as reliability, correctness, and efficiency—are known as external quality. While the quality aspects that relate to the development organisation or team, such as maintainability, readability, ‘understandability’ and complexity—are known as internal quality (see Figure 1).

Most developers understand that there is a difference between the quality aspects that concern themselves versus the quality aspects that impact the users. Once they see it in a visual form, like shown in Figure 1, they are able to appreciate the differences much better. Still, they are normally not able to perceive how they are related to each other. However, experienced architects understand that internal quality has considerable impact on external quality! This link is hard to see, and the aspects of internal quality are easy to ignore, since it is often not very visible. The picture shows an iceberg to illustrate this point: only a small part of the iceberg is visible above the water, while 1/8th to 1/10th of it is submerged beneath. But they are interlinked! And the software project ‘ship’ can sink if we underestimate the impact of poor internal quality by superficially focusing only on external quality!

Most development teams and project managers, in the name of being pragmatic, focus only on the external quality aspects. They focus on testing, which can be used for improving external quality. They run dynamic analysis tools and profiling tools to find and fix problems, or improve the software on the externally perceivable quality aspects. But what they don’t realise is that most quality problems are internal to the software—and unless internal quality is good, external quality cannot be, even if it appears to be; poor internal quality can derail the project. This is analogous to the captain of the Titanic, who underestimated the impact of the iceberg because it looked small, but the large part of the iceberg that was submerged in the water sank the ship.

Internal (structural) quality affects external (behavioural) quality – this is the most important axiom in software quality. In the name of being pragmatic, or just plain ignorance, many managers disregard internal quality. Recently, a manager who is a big fan of agile methods made a startling statement: “Refactoring is rework, and hence is a total waste of time and resources. We should get it right the first time!” Superficially, the statement makes sense—why refactor and rework when you can get it right the first time? But if you reflect on it, you’ll see how ridiculous this statement is, especially in the context of agile methods.

The proponents of agile methods have a very good understanding of the dynamics of software development, and what goes into making high-quality software. One of the basic

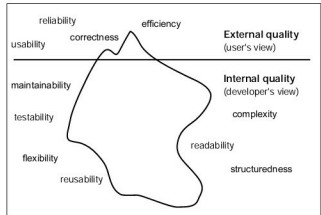


Figure 1: The software quality iceberg (internal vs external quality)

tenets of agile methods is that software evolves by adapting the software to ever-changing requirements. With every incoming change, the existing structure needs to be improved and adapted, which is achieved using refactoring. In fact, one of the principles of agile methods is that “...continuous attention to technical excellence and good design enhances agility.” With this, instead of up-front investment in design, good design is achieved through continuous refactoring. If you take out refactoring, and keep making changes to add more functionality, that will result in rotten code. To return to our earlier discussion on how internal quality influences external quality, the rotten code will ensure that the project is a failure. “We should get it right the first time,” is the approach with up-front investment in design, the approach that the traditional waterfall model encourages. The agile approach does not recommend up-front investment in design, and instead relies on good design that emerges because of continuous refactoring. Hence, refactoring is an integral part of agile methods, and should not be taken away because it is ‘needless rework’!

To be successful, software projects (open source or enterprise) need to focus on internal quality. That is the reason why design quality is of paramount importance. Structure gives rise to behaviour: ‘external quality’ is emergent in nature and it emerges from the internal structure of the software. Hence, the way to good external quality is to focus on and improve the internal quality of the software. **END** 

References:

[1] Titanic Dilemma: The Seen Versus the Unseen: <http://blog.castssoftware.com/titanic-dilemma-the-seen-versus-the-unseen/>

By: S G Ganesh

The author works for Siemens (Corporate Research & Technologies), Bangalore. You can reach him at sgganesh@gmail.com.